Fun Run - ECS Architecture & Technical Design Document

This document establishes the architectural framework and progressive development plan for the final project: a multiplayer 2D racing platformer based on the Entity Component System (ECS) pattern. The game is implemented using C++, incorporating SDL3 for windowing and input, SDL3_image for graphics, and Box2D for physics simulation.

1. Core Gameplay Gimmick: Personal Gravity Shift

To satisfy the unique feature requirements, the game introduces a **Personal Gravity Shift** mechanic. Each player entity can independently invert their localized gravity vector. When triggered, the specific player falls upward toward the ceiling, allowing them to traverse alternative paths, avoid ground-based obstacles, or dodge enemy projectiles. This behavior operates completely independently for each player, meaning one player can be running on the ceiling while opponents remain on the floor.

2. ECS Components Definition

Components are pure data structures. They contain no behavior or logic. Below is the technical specification of the components required for the simulation:

Component Name	Description & Purpose	Data Fields Included
TransformComponent	Stores the absolute spatial coordinates of the entity. Synchronized with the physics simulation.	SDL_FPoint position float rotation_degrees
DrawableComponent	Contains texture rendering properties and spreadsheet layout configurations for animation states.	SDL_Texture* texture SDL_FRect src_rect SDL_FRect dest_dimensions int flip_flags

Component Name	Description & Purpose	Data Fields Included
PhysicsBodyComponent	Bridges the ECS entity to the underlying Box2D rigid body instance.	b2BodyId body_id b2ShapeId shape_id bool is_grounded
PlayerInputComponent	Stores the raw intent/actions polled from the player hardware controls during the current frame.	bool move_right bool move_left bool jump_pressed bool use_item_pressed bool gravity_shift_pressed
PlayerStateComponent	Maintains status conditions, speed modifiers, race rankings, and current inventory.	int lives float current_speed_multiplier int current_powerup_id bool is_eliminated
GravityShiftComponent	Tracks status and limitations regarding the personal gravity gimmick.	bool is_inverted float cooldown_timer_ms float maximum_shift_duration_ms
SensorAreaComponent	Flags an entity as a non-colliding trigger region (e.g., item boxes, finish lines).	int sensor_type_id bool is_triggered_this_frame
TrapComponent	Appended to active projectiles or hazards to process defensive/offensive penalties.	float speed_penalty_factor float stun_duration_seconds uint64_t owner_entity_id

3. ECS Entities Blueprint

Entities are unique IDs acting as containers for specific combinations of components:

- **Player Entity:** Composed of TransformComponent, DrawableComponent, PhysicsBodyComponent, PlayerInputComponent, PlayerStateComponent, and GravityShiftComponent.
- **Environment / Platform Entity:** Composed of TransformComponent, DrawableComponent, and PhysicsBodyComponent (configured as a static body type).
- **Item Box Mystery Entity:** Composed of TransformComponent, DrawableComponent, and SensorAreaComponent.
- **Projectile / Active Hazard Entity:** Composed of TransformComponent, DrawableComponent, PhysicsBodyComponent (kinematic/dynamic), and TrapComponent.
- **Camera Entity:** Composed of TransformComponent (used to store coordinate anchors for screen tracking calculations).

4. ECS Systems Design

Systems process entities that possess matching component signatures. They run once per frame loop execution:

- **InputSystem:** Loops through game peripheral events (via SDL_PollEvent). Maps assigned hardware configurations to the respective player indices, populating their PlayerInputComponent data fields.
- **PlayerControllerSystem:** Evaluates PlayerInputComponent and PlayerStateComponent. Applies forces or sets linear velocities within the Box2D engine. If the gravity shift condition is met, it updates the GravityShiftComponent and modifies the local body properties.
- **PhysicsSystem:** Updates the physics simulation environment using Box2D's step command. Extracts updated position vectors from the active b2BodyId handles and writes them directly into the corresponding TransformComponent.
- **SensorCollisionSystem:** Iterates through Box2D contact and sensor event queues. Identifies player intersections with item triggers, updates inventory attributes, and signals deletion of used item boxes.
- **DamageSystem:** Processes entity overlapping states involving harmful traps or boundaries. Modifies performance variables or triggers respawn logic.
- **CameraSystem:** Computes the maximum forward positioning coordinate across all non-eliminated players. Anchors the camera's viewport coordinate to track the lead racer,

establishing culling boundaries for slower players.

- **RenderSystem:** Clears the current rendering target via SDL. Determines structural offsets using the active Camera coordinates, passing visual boundaries down to SDL_RenderTexture for execution.